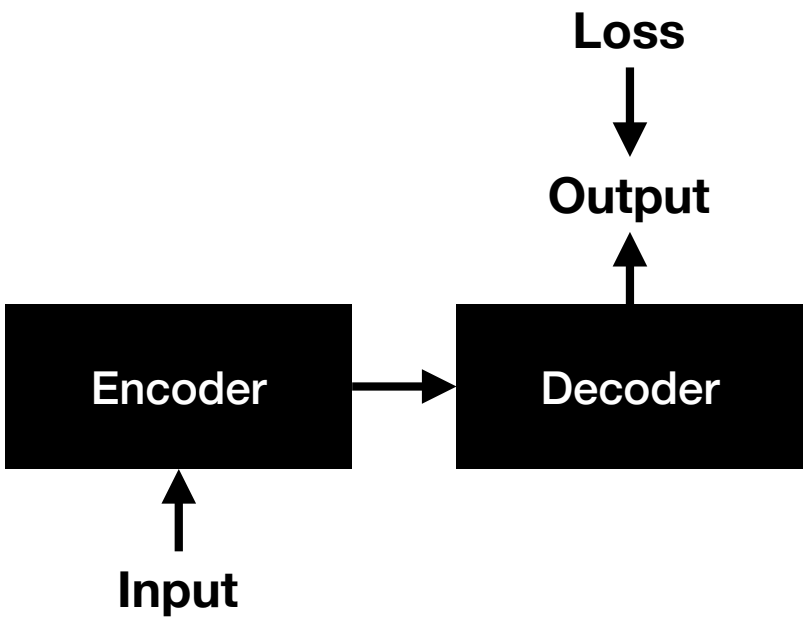
Raw Data

Pre-training Encoder-Decoders

Cornell CS 5740: Natural Language Processing
Yoav Artzi, lm-class-v2025.1

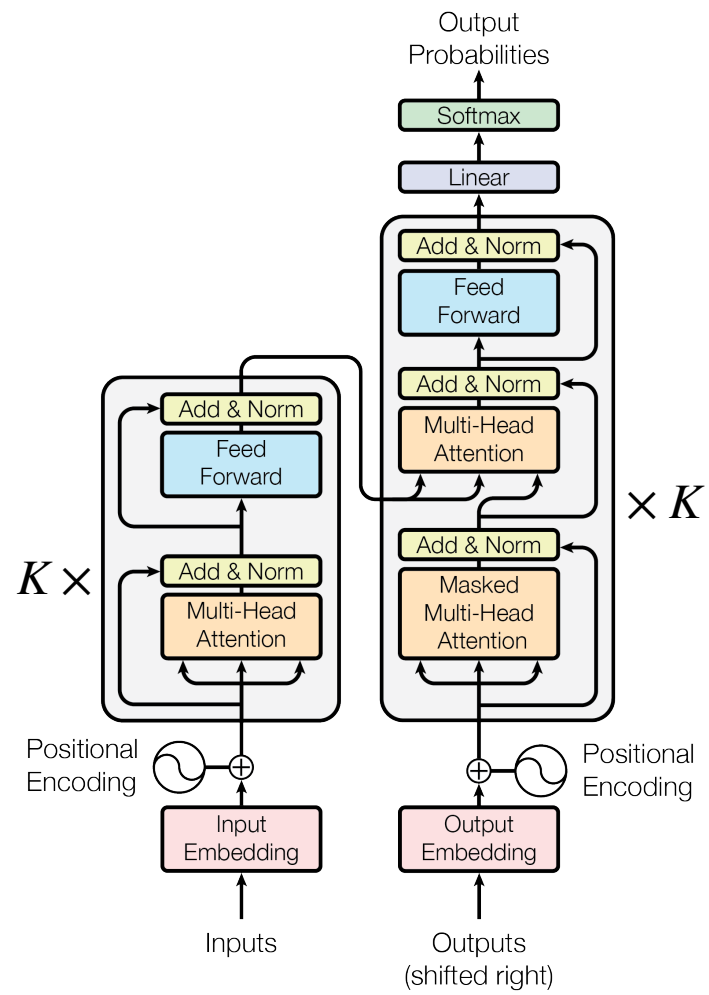
Encoder-decoder

- The model is composed of two components
- Bidirectional **encoder** to process the input
- Autoregressive **decoder** to generate output
- Training is usually done with loss on the output
  - Propagates into the decoder and through it to the encoder



Encoder-decoder Transformer

- So far:
  - Encoder: processes a complete sentence to get representations
  - Decoder: generates a sentence
- Let's combine both to an encoder-decoder



[Vaswani et al. 2017]

Encoder-decoder Transformer

The Encoder

$\text{TransformerBlock}^{\text{k}}(\mathbf{u}_1, \ldots, \mathbf{u}_\text{n})$

$$\mathbf{Q}^{(l)} = \text{Linear}_\text{q}^{(l)}([\mathbf{u}_1 \cdots \mathbf{u}_\text{n}])$$

$$\mathbf{K}^{(l)} = \text{Linear}_\text{k}^{(l)}([\mathbf{u}_1 \cdots \mathbf{u}_\text{n}])$$

$$\mathbf{V}^{(l)} = \text{Linear}_\text{v}^{(l)}([\mathbf{u}_1 \cdots \mathbf{u}_\text{n}])$$

$$\mathbf{Z} = \text{LN}(\text{Linear}_\text{z}([\text{SelfAttn}(\mathbf{Q}^{(1)}, \mathbf{K}^{(1)}, \mathbf{V}^{(1)}); \cdots; \text{SelfAttn}(\mathbf{Q}^{(L)}, \mathbf{K}^{(L)}, \mathbf{V}^{(L)})]) + [\mathbf{u}_1 \cdots \mathbf{u}_n])$$

$$[\mathbf{e}_1^k \cdots \mathbf{e}_n^k] = \text{LN}(\text{Linear}_2(\text{GELU}(\text{Linear}_1(\mathbf{Z}))) + \mathbf{Z})$$

$$\mathbf{x}_j = \phi^E(x_j) + \phi_p^E(j), j = 1, \ldots, n$$

$$[\mathbf{e}_1^1 \cdots \mathbf{e}_n^1] = \text{TransformerBlock}^1(\mathbf{x}_1, \ldots, \mathbf{x}_\text{n})$$

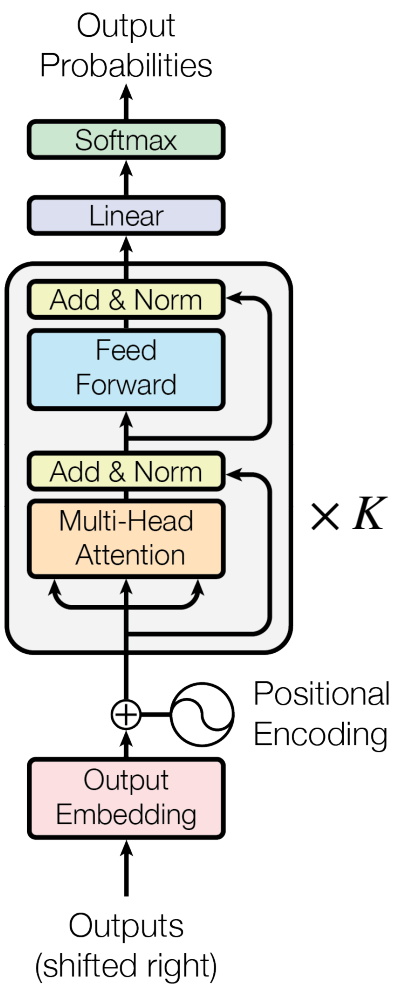
$$[\mathbf{e}_1^2 \cdots \mathbf{e}_n^2] = \text{TransformerBlock}^2(\mathbf{e}_1^1, \ldots, \mathbf{e}_\text{n}^1)$$

$$\ldots$$

$$[\mathbf{e}_1^k \cdots \mathbf{e}_n^k] = \text{TransformerBlock}^\text{k}(\mathbf{e}_1^{\text{k}-1}, \ldots, \mathbf{e}_\text{n}^{\text{k}-1})$$

$$\ldots$$

$$[\mathbf{e}_1^K \cdots \mathbf{e}_n^K] = \text{TransformerBlock}^\text{K}(\mathbf{e}_1^{\text{K}-1}, \ldots, \mathbf{e}_\text{n}^{\text{K}-1})$$



Self-attention reminder

$$\text{SelfAttn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{QK}/\sqrt{d_k})\mathbf{V}$$

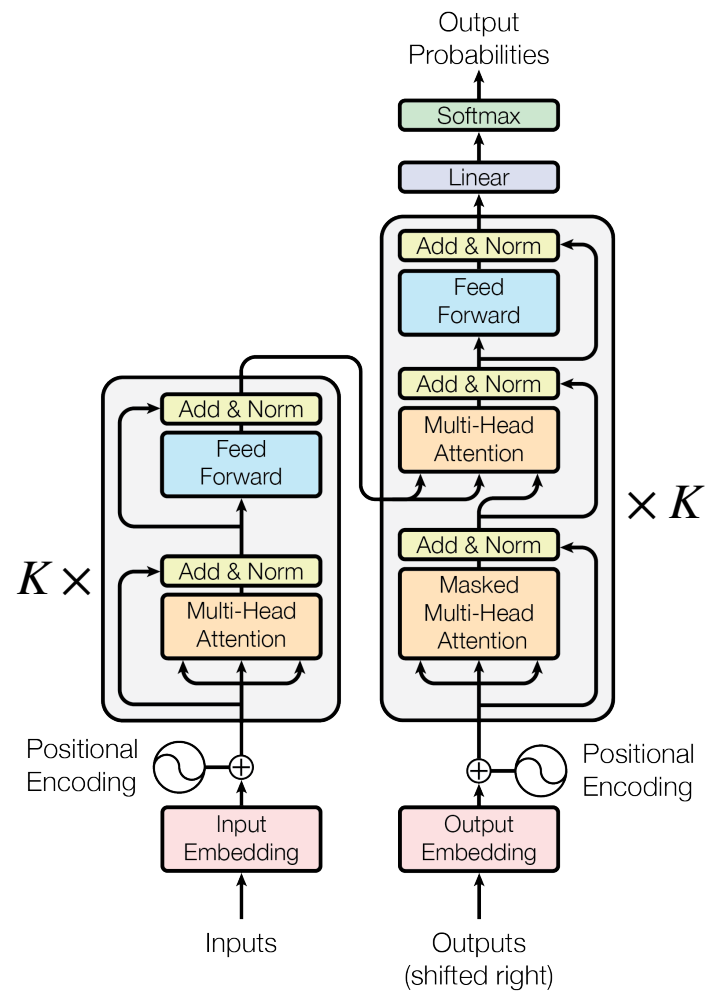
- No per-token prediction
- Changed hidden states to $\mathbf{e}$

[Vaswani et al. 2017]

Encoder-decoder Transformer

The Decoder

- Decoder is identical to the regular decoder
- Except that we need to attend to the hidden states in the input
- So: we add another multi head attention to the Transformer Block



[Vaswani et al. 2017]

Encoder-decoder Transformer

The Decoder Block

$$\text{TransformerBlockDecoder}^{k}(\mathbf{u}_1, \ldots, \mathbf{u}_i, \mathbf{e}_1^k, \ldots, \mathbf{e}_n^k)$$

$$\mathbf{q}^{(l)} = \text{Linear}_q^{(l)}(\mathbf{u}_i)$$

$$\mathbf{K}^{(l)} = \text{Linear}_k^{(l)}([\mathbf{u}_1 \cdots \mathbf{u}_i])$$

$$\mathbf{V}^{(l)} = \text{Linear}_v^{(l)}([\mathbf{u}_1 \cdots \mathbf{u}_i])$$

$$\mathbf{z} = \text{LN}(\text{Linear}_z([\text{SelfAttn}(\mathbf{q}^{(1)}, \mathbf{K}^{(1)}, \mathbf{V}^{(1)}); \cdots; \text{SelfAttn}(\mathbf{q}^{(L)}, \mathbf{K}^{(L)}, \mathbf{V}^{(L)})]) + \mathbf{u}_i)$$

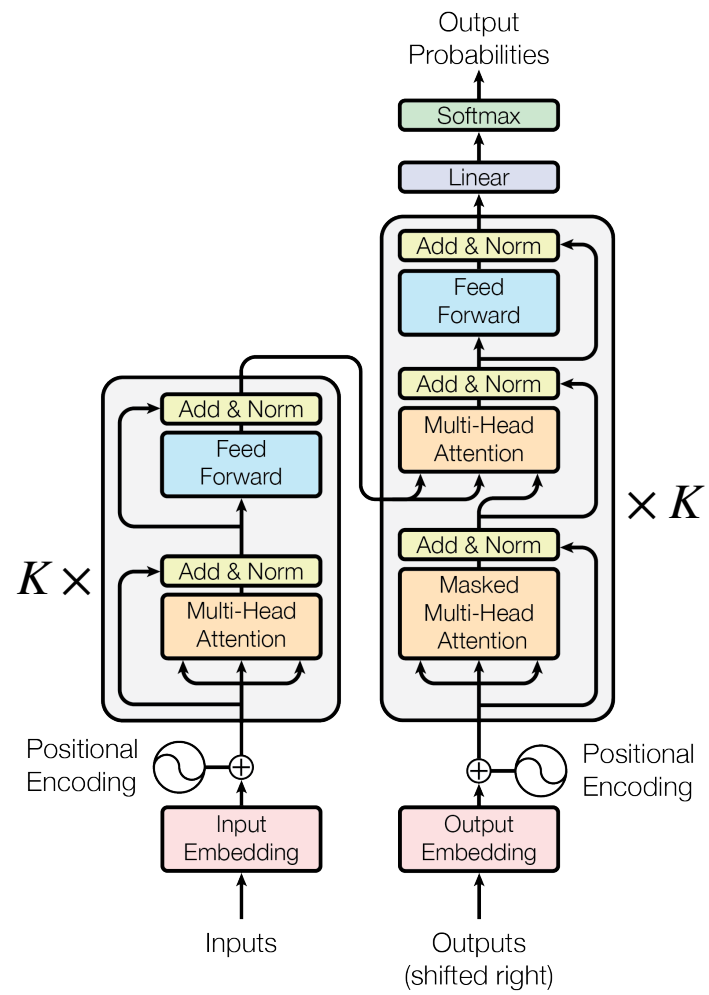
$$\mathbf{q}^{(l)} = \text{Linear}_{q'}^{(l)}(\mathbf{z})$$

$$\mathbf{K}^{(l)} = \text{Linear}_{k'}^{(l)}([\mathbf{e}_1 \cdots \mathbf{e}_n])$$

$$\mathbf{V}^{(l)} = \text{Linear}_{v'}^{(l)}([\mathbf{e}_1 \cdots \mathbf{e}_n])$$

$$\mathbf{z} = \text{LN}(\text{Linear}_{z'}([\text{SelfAttn}(\mathbf{q}^{(1)}, \mathbf{K}^{(1)}, \mathbf{V}^{(1)}); \cdots; \text{SelfAttn}(\mathbf{q}^{(L)}, \mathbf{K}^{(L)}, \mathbf{V}^{(L)})]) + \mathbf{z})$$

$$\mathbf{d}_i^k = \text{LN}(\text{Linear}_2(\text{GELU}(\text{Linear}_1(\mathbf{z}))) + \mathbf{z})$$



[Vaswani et al. 2017]

Encoder-decoder Transformer

The Decoder

$$\mathbf{y}_i = \phi^D(y_i) + \phi_p^D(i)$$

$$\mathbf{d}_i^1 = \text{TransformerBlock}^1(\mathbf{x}_1, \ldots, \mathbf{x}_\text{i}, \mathbf{e}_1^1, \ldots, \mathbf{e}_\text{n}^1)$$

$$\mathbf{d}_i^2 = \text{TransformerBlock}^2(\mathbf{d}_1^1, \ldots, \mathbf{d}_\text{i}^1, \mathbf{e}_1^2, \ldots, \mathbf{e}_\text{n}^2)$$

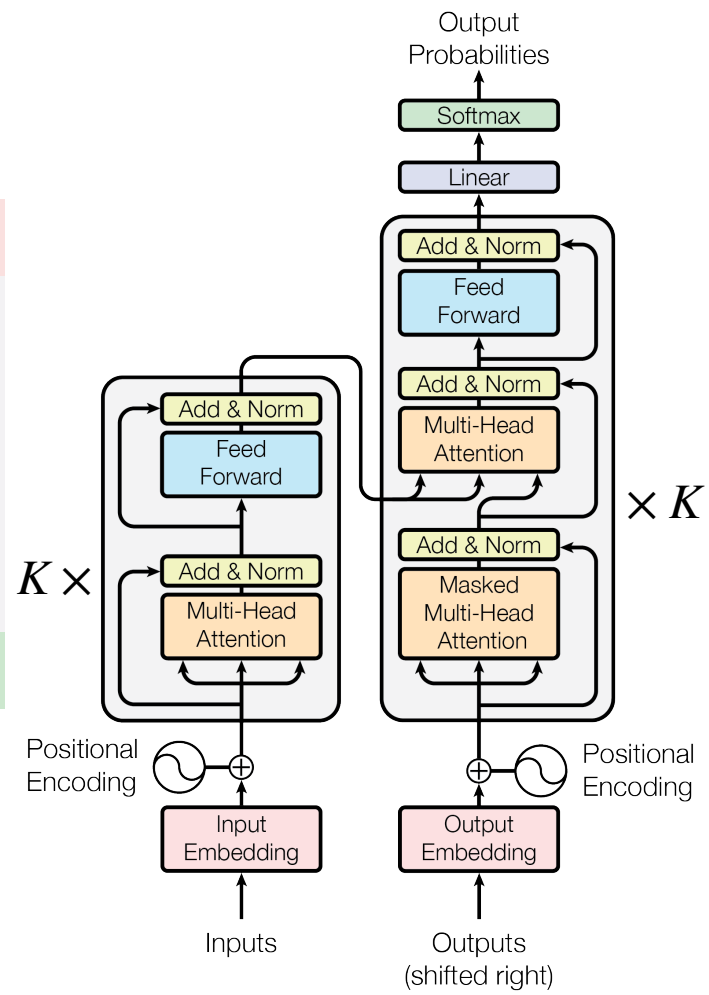
$$\ldots$$

$$\mathbf{d}_i^k = \text{TransformerBlock}^\text{k}(\mathbf{d}_1^{\text{k}-1}, \ldots, \mathbf{d}_\text{i}^{\text{k}-1}, \mathbf{e}_1^\text{k}, \ldots, \mathbf{e}_\text{n}^\text{k})$$

$$\ldots$$

$$\mathbf{d}_i^K = \text{TransformerBlock}^\text{K}(\mathbf{d}_1^{\text{K}-1}, \ldots, \mathbf{d}_\text{i}^{\text{K}-1}, \mathbf{e}_1^\text{K}, \ldots, \mathbf{e}_\text{n}^\text{K})$$

$$p(y_{i+1} \mid y_1, \ldots, y_i, \bar{x}) = \text{softmax}(\text{Linear}_{\mathscr{V}}(\mathbf{h}_\text{i}^\text{K}))$$

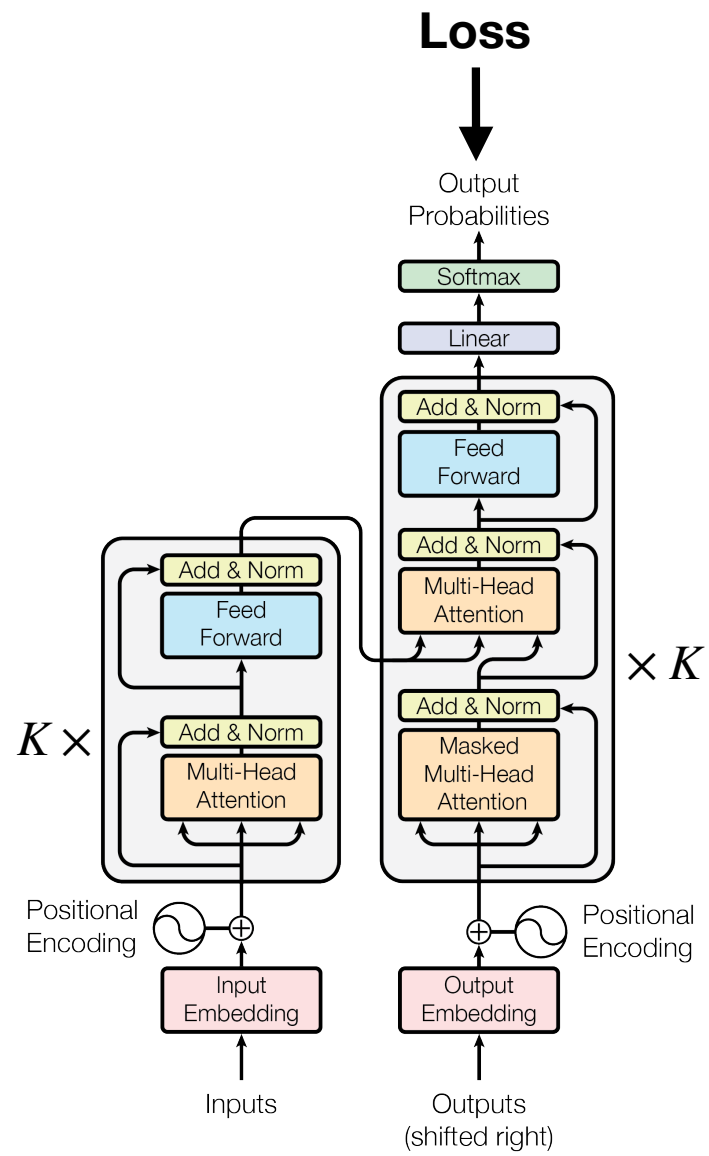


[Vaswani et al. 2017]

Encoder-decoder Transformer

With Transformers

- Bidirectional **encoder** to process the input
- Autoregressive **decoder** to generate output
- Why does this structure make sense?



[Vaswani et al. 2017]

Encoder-decoder Pre-training

The BART Recipe

- An encoder-decoder (sequence-to-sequence) pre-trained model
- Extends the BERT approach to encoder-decoder

[Lewis et al. 2019]

BART

BERT Reminder

- Encoder-only Transformer
- Trained on raw data
- Two self-supervised objects:
  - Masked LM
  - Next-sentence prediction
- Transformed the NLP task landscape — if you have enough data, fine-tuned BERT works really well

[Lewis et al. 2019]

BART

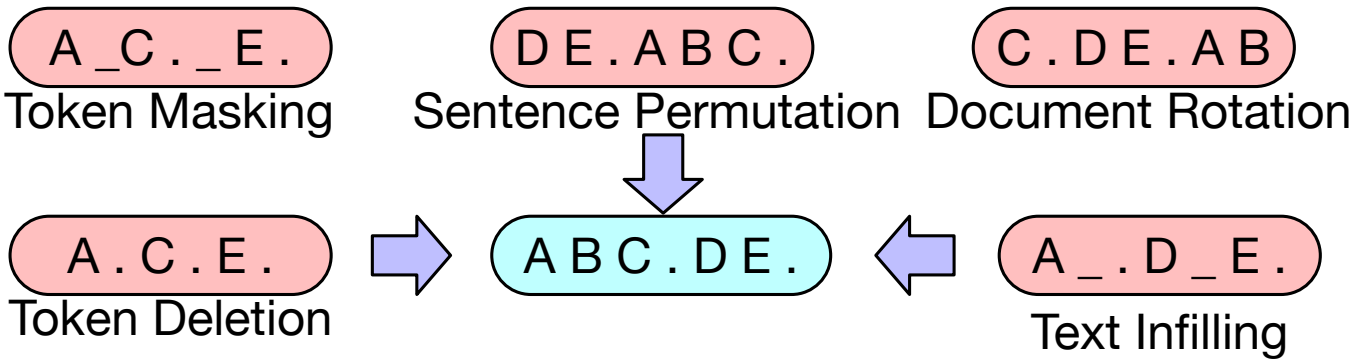
- How does the BERT learning approach adapts to an encoder-decoder architecture?
  - Output is generated by decoder, and the loss is on the output
  - Input is a sequence of tokens

[Lewis et al. 2019]

BART

Denoising Self-supervised Objective

- Corrupt the input following five different recipes



- Try to recover the pre-corrupted input by generating it using the decoder
- Train on a lot of raw text data, just like with BERT
- How to compute the loss? Loss can be computed using “teacher forcing” just like with language modeling, but on the decoder only

[Lewis et al. 2019]

BART

What Do We Get?

- BERT: a pre-trained encoder
- BART: pre-trained decoder and encoder
  - Can use both
  - Or can use only the decoder wherever we would use BERT

[Lewis et al. 2019]

BART

How to Use?

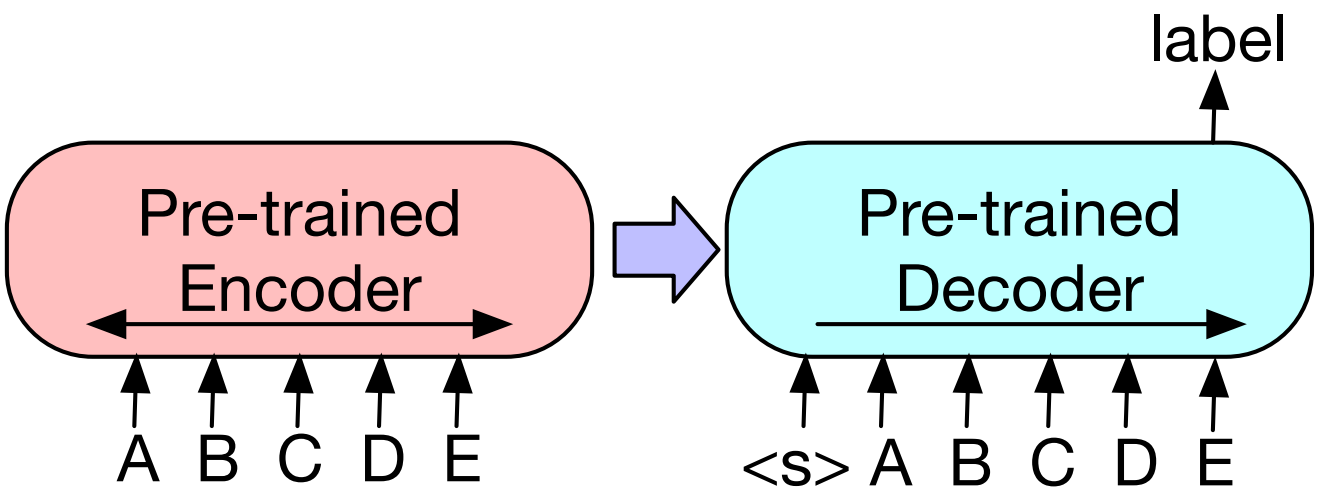
- Similar to BERT: fine-tune for the end task
- How can we use for classification?

[Lewis et al. 2019]

BART

Classification

- The input is given the encoder
- The same input is force-decoded in the decoder via “teacher forcing”
- The representation from the final decoder hidden state is given to a classification head



[Lewis et al. 2019]

BART

Performance

- Can do anything that BERT does
- But can also do generation tasks (e.g., summarization)

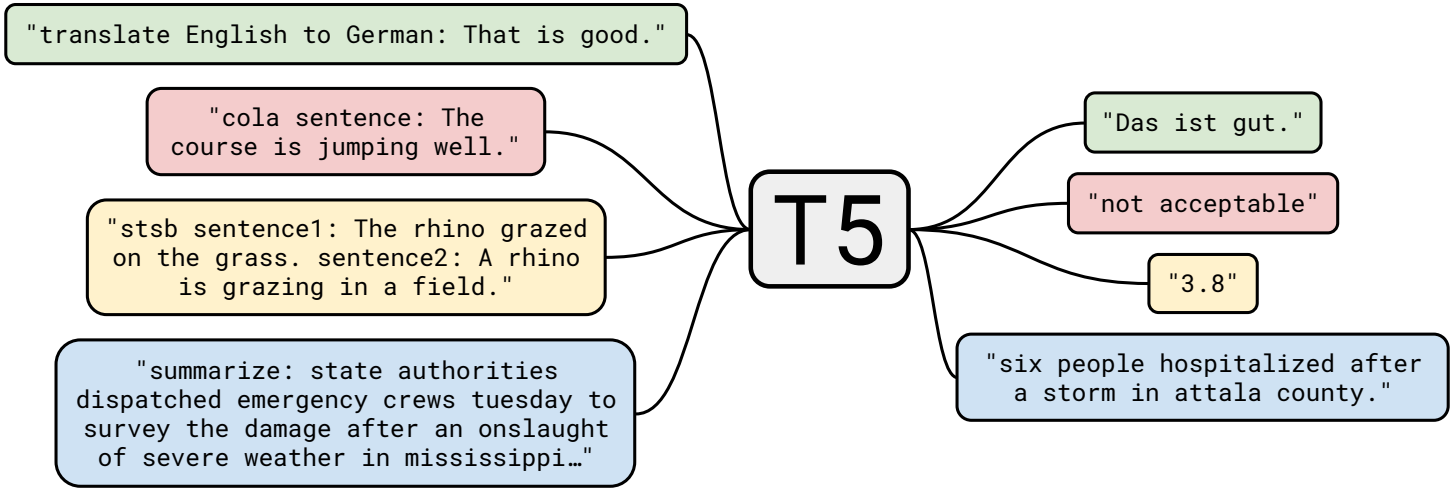
Model	SQuAD 1.1 F1	MNLI Acc	ELI5 PPL	XSum PPL	ConvAI2 PPL	CNN/DM PPL
BERT Base (Devlin et al., 2019)	88.5	**84.3**	-	-	-	-
BART Base						
w/ Token Masking	90.4	84.1	25.05	7.08	11.73	6.10
w/ Token Deletion	90.4	84.1	24.61	6.90	11.46	5.87
w/ Text Infilling	**90.8**	84.0	24.26	**6.61**	**11.05**	5.83
w/ Document Rotation	77.2	75.3	53.69	17.14	19.87	10.59
w/ Sentence Shuffling	85.4	81.5	41.87	10.93	16.67	7.89
w/ Text Infilling + Sentence Shuffling	**90.8**	83.8	24.17	6.62	11.12	**5.41**

[Lewis et al. 2019]

Encoder-decoder Pre-training

The T5 Recipe

- Concurrent and similar to BART
- Also adopted the text-to-text approach for all NLP tasks



[Raffel et al. 2019]

T5

Pretraining

- Pretraining is similar to the denoising objective of BART:
  - Input: text with gaps (phrases removed)
  - Output: sequence of phrases to fill the gaps

Original text

Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

[Raffel et al. 2019]

T5

Results

- T5 was trained on one of the first very large corpora: 750GB of text, with pre-training using 2^{35} tokens
- First to show the impact of data scale
- Why did they repeat the smaller scale datasets?

Number of tokens	Repeats	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Full data set	0	**83.28**	**19.24**	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
2^{29}	64	**82.87**	**19.19**	**80.97**	**72.03**	**26.83**	**39.74**	**27.63**
2^{27}	256	82.62	**19.20**	79.78	69.97	**27.02**	**39.71**	27.33
2^{25}	1,024	79.55	18.57	76.27	64.76	26.38	39.56	26.80
2^{23}	4,096	76.34	18.33	70.92	59.29	26.37	38.84	25.81

[Raffel et al. 2019]

BART and T5

Takeaways

- BART and T5 are very useful for all sorts of sequence-to-sequence tasks with language
  - T5 comes in different sizes
  - There are various customization (e.g., CodeT5)
- Extended the generalizations conclusions from BERT, and demonstrated the impact of data scale

Acknowledgements

- Slides are based on slides from UT Austin CS 388 by Greg Durrett